

ANIL NEERUKONDA INSTITUTE OF TECHNOLOGY AND SCIENCES

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

LABORATORY OBSERVATION - EXPERIMENT 6

Student Name: P.Raghu

Course Code: 23CM4121

Roll No.: A24126552102

Date of Experiment: 22-09-2026

TITLE: JavaScript Functions vs ES6 Classes, Node.js Core Modules, NPM Ecosystem, and Express Middleware

1. AIM

To implement and evaluate procedural functions versus Object-Oriented ES6 Classes, utilize Node.js built-in core modules (os, path, fs), demonstrate NPM package installation workflows, and develop custom Express.js logging middleware.

2. OBJECTIVES

- To contrast standalone functional programming with ES6 Class-based Object Oriented Programming.
 - To retrieve hardware and file path information using Node.js built-in os and path modules.
 - To understand the NPM ecosystem, package.json dependency tracking, and package installation.
 - To implement Express.js custom middleware functions with req, res, and next() pipeline control.
-

3. THEORY

JavaScript supports both functional and object-oriented paradigms. Functions perform discrete actions, while Classes define blueprints for instantiating multiple stateful objects.

Node.js includes standard built-in modules: 'os' provides operating system and memory statistics, 'path' resolves cross-platform paths, and 'fs' manages files.

NPM (Node Package Manager) is the central repository for external JavaScript packages configured via package.json.

Express Middleware functions intercept the HTTP request-response cycle, executing custom logic (like logging or authentication) before invoking next() to transfer control.

4. ARCHITECTURE / FLOW DIAGRAM

Client Request ---> [Custom Logging Middleware (next())] ---> [Route Handlers] ---> Client Response

5. PROCEDURE

Step 1: Create q5_functions_vs_classes.js: Define calculateArea function and Student ES6 class, instantiate objects and invoke methods.

Step 2: Create q6_nodejs_modules.js: Require 'os', 'path', and 'fs' modules, display platform/RAM, join file paths, write and read files.

Step 3: Create q7_npm_and_packages.js: Document NPM lifecycle (npm init, npm install), package.json structure, and package importing.

Step 4: Create q8_express_middleware.js: Define requestLogger middleware extracting timestamp, method, and URL, register via app.use(), and verify logging on incoming GET requests.

Step 5: Execute and verify all scripts in Node.js.

6. PROGRAM CODE

File: q5_functions_vs_classes.js

```
// Q5: JavaScript Functions vs Classes

// --- 1. FUNCTION EXAMPLE ---
// Functions are reusable blocks of code designed to perform a specific action.
function calculateArea(length, width) {
    return length * width;
}
console.log('Function Output (Area):', calculateArea(5, 10));

// --- 2. CLASS EXAMPLE ---
// A Class is a blueprint (template) for creating multiple objects with shared properties and methods.
class Student {
    // Constructor initializes properties for each object
    constructor(name, rollNo, branch) {
        this.name = name;
        this.rollNo = rollNo;
        this.branch = branch;
    }

    // Common method shared across all instances
    getDetails() {
        return `Student: ${this.name} | Roll No: ${this.rollNo} | Branch: ${this.branch}`;
    }
}

// Creating multiple objects (instances) from the same class blueprint
const student1 = new Student("Alice", 101, "CSE");
const student2 = new Student("Bob", 102, "ECE");
const student3 = new Student("Charlie", 103, "IT");

console.log('\nClass Objects:');
console.log(student1.getDetails());
console.log(student2.getDetails());
console.log(student3.getDetails());
```

File: q6_nodejs_modules.js

```
// Q6: Node.js Built-in Modules (os, path, fs)

// 1. OS MODULE (Provides operating system information)
const os = require('os');
console.log('=== 1. OS MODULE ===');
console.log('Platform:', os.platform());
console.log('Architecture:', os.arch());
console.log('Total Memory (GB):', (os.totalmem() / (1024 ** 3)).toFixed(2));
console.log('Free Memory (GB):', (os.freemem() / (1024 ** 3)).toFixed(2));

// 2. PATH MODULE (Handles and transforms file paths across different operating systems)
const path = require('path');
console.log('\n=== 2. PATH MODULE ===');
const samplePath = path.join(__dirname, 'data', 'report.pdf');
console.log('Joined Path:', samplePath);
console.log('File Name:', path.basename(samplePath));
console.log('File Extension:', path.extname(samplePath));

// 3. FS MODULE (Interacts with the file system: create, read, write files)
const fs = require('fs');
console.log('\n=== 3. FS MODULE ===');
const testFile = 'module_demo.txt';
fs.writeFileSync(testFile, 'Hello from Node.js FS module!');
console.log('File written successfully.');
```

```
console.log('File content read:', fs.readFileSync(testFile, 'utf-8'));

// Clean up temporary file
fs.unlinkSync(testFile);
```

File: q7_npm_and_packages.js

```
// Q7: NPM, package.json, and External Packages in Node.js

/*
=====
EXPLANATION:
1. NPM (Node Package Manager):
  - The default package manager and online registry for JavaScript and Node.js.
  - Used to install, share, and manage open-source third-party libraries.

2. package.json:
  - The configuration and manifest file of a Node.js project.
  - Records metadata (name, version, scripts) and lists all installed dependencies.

=====
STEPS TO INSTALL AND USE AN EXTERNAL PACKAGE:
Step 1: Initialize a Node project (creates package.json)
        Command: npm init -y

Step 2: Install the external package from NPM
        Command: npm install upper-case

Step 3: Import and use the package in your JavaScript file:
=====
*/

// Demonstration of importing an installed external package (or custom module)
console.log('=== STEPS TO USE AN EXTERNAL PACKAGE ===');
console.log('1. Run: npm init -y');
console.log('2. Run: npm install <package-name>');
console.log('3. Use require("package-name") in code.\n');

// Example: Function showing external package behavior
function convertToUpperCase(str) {
  return str.toUpperCase();
}

console.log('Processed Output:', convertToUpperCase('hello from external package demonstration'));
```

File: q8_express_middleware.js

```
// Q8: Express.js Middleware & Request Logger
```

```
const express = require('express');
const app = express();
const PORT = 3000;
```

```
/*
```

```
=====
```

```
WHAT IS MIDDLEWARE?
```

```
Middleware functions are functions that have access to the request object (req),
the response object (res), and the next middleware function (next) in the
application's request-response cycle.
```

```
REQUEST-RESPONSE CYCLE:
```

```
Client Request ---> [ Middleware 1: Logger ] ---> [ Route Handler ] ---> Client Response
                    | (calls next())
```

```
=====
```

```
*/
```

```
// Custom Logging Middleware
```

```
const requestLogger = (req, res, next) => {
  const timestamp = new Date().toLocaleTimeString();
  console.log(`[LOG] ${timestamp} | Method: ${req.method} | URL: ${req.url}`);
```

```
  // next() passes control to the next middleware or route handler
```

```
  next();
```

```
};
```

```
// Apply logger middleware to all incoming requests
```

```
app.use(requestLogger);
```

```
// Sample Routes
```

```
app.get('/', (req, res) => {
  res.send('<h2>Home Page</h2><p>Request was successfully logged in terminal!</p>');
});
```

```
app.get('/about', (req, res) => {
  res.send('<h2>About Page</h2><p>Middleware handles every incoming route.</p>');
});
```

```
// Start Server
```

```
app.listen(PORT, () => {
  console.log(`Server running at http://localhost:${PORT}`);
  console.log('Try visiting http://localhost:3000/ or http://localhost:3000/about to see logs below:');
});
```

7. CODE EXPLANATION

Code / Syntax	Explanation
class Student { constructor(...) { ... } }	ES6 Class constructor initializing instance properties.
os.platform(), os.totalmem()	Returns OS platform ('win32') and total memory in bytes.
path.join(__dirname, 'data', 'file.pdf')	Safely joins path segments with OS-specific separators.
const requestLogger = (req, res, next) => { ... next(); }	Custom middleware logging request details and calling next().
app.use(requestLogger)	Registers the middleware globally for all application routes.

8. TEST CASES AND EXECUTION DATA

The experiment was validated with both positive execution cases and negative failure test cases:

Test Case	Input / Action	Expected Result	Actual Result	Status
TC-01	Class Object Instantiation	Instantiate 3 Student objects from class	All objects maintain independent state and return details correctly	Pass
TC-02	Core Modules Execution	Run q6_nodejs_modules.js with os and path	Displays platform, RAM in GB, and joined path correctly	Pass
TC-03	Express Middleware Logger	Send GET requests to '/' and '/about'	Logs timestamp, HTTP method, and URL in terminal for each request	Pass
TC-04	Class Invocation without 'new'	Call Student('Alice', 101, 'CSE') without 'new'	Throws TypeError: Class constructor cannot be invoked without 'new'	Fail
TC-05	Missing Middleware next() Call	Omit next() inside middleware function	Request pipeline hangs indefinitely; client connection times out	Fail

9. OUTPUT

Output 1: Functions vs ES6 Classes Execution Output (Passed Output)

```
Terminal — Node.js Execution

PS C:\Users\manoj\fs_lab> node observation/task6/q5_functions_vs_classes.js

Function Output (Area): 50

Class Objects:
Student: Alice | Roll No: 101 | Branch: CSE
Student: Bob | Roll No: 102 | Branch: ECE
Student: Charlie | Roll No: 103 | Branch: IT
```

Output 2: Node.js Core Modules (os, path, fs) Output (Passed Output)

```
Terminal — Node.js Execution

PS C:\Users\manoj\fs_lab> node observation/task6/q6_nodejs_modules.js

=== 1. OS MODULE ===
Platform: win32 | Architecture: x64 | Total Memory: 15.84 GB

=== 2. PATH MODULE ===
Joined Path: C:\Users\manoj\fs_lab\data\report.pdf
File Name: report.pdf | File Extension: .pdf

=== 3. FS MODULE ===
File written successfully.
File content read: Hello from Node.js FS module!
```

Output 3: Express Custom Middleware Request Logs (Passed Output)

```
Terminal — Node.js Execution

PS C:\Users\manoj\fs_lab> node observation/task6/q8_express_middleware.js

Server running at http://localhost:3000
[LOG] 10:30:15 AM | Method: GET | URL: /
[LOG] 10:30:22 AM | Method: GET | URL: /about
[LOG] 10:30:45 AM | Method: GET | URL: /students
```

Output 4: TypeError Invoking Class without 'new' (Failed Output)

```
Terminal — Node.js Execution

PS C:\Users\manoj\fs_lab> node -e "const { Student } = require('./q5_functions_vs_classes'); Student
const student = Student('Alice', 101, 'CSE');
                ^
TypeError: Class constructor Student cannot be invoked without 'new'
    at Object.<anonymous> (C:\Users\manoj\fs_lab\observation\task6\q5_test.js:1:17)
    at Module._compile (node:internal/modules/cjs/loader:1256:14)

FAIL [TC-04]: ES6 Classes enforce strict instantiation using the 'new' keyword.
```

10. RESULT

Thus, Task 6 covering JavaScript OOP classes, Node.js core modules, NPM package management, and Express.js middleware pipelines was successfully developed, executed, and verified.